

Roadmap pour créer une API REST avec Spring Boot

1 Préparation et configuration du projet - Créer le projet Spring Boot (Spring Initializr ou IntelliJ) - Dépendances : Spring Web, Spring Data JPA, Base de données (H2/PostgreSQL/MySQL), Lombok (optionnel), Spring Boot DevTools (optionnel) - Configurer `application.properties` ou `application.yml` : URL, user, mot de passe, port, logging

2 Définir le modèle de données - Créer les entités JPA (`@Entity`) : CaveStock, CaveSupplier - Définir relations : `@OneToMany`, `@ManyToOne` - Ajouter contraintes : `@NotNull`, `@Size`, `@Column(unique=true)`

3 Créer la couche repository - Interfaces qui étendent `JpaRepository` ou `CrudRepository` - Exemples de requêtes personnalisées : `List<CaveStock> findByGenre(String genre);`

4 Créer la couche service - Classes `@Service` pour la logique métier - Méthodes typiques : `getAllStocks()`, `getStockById()`, `createStock()`, `updateStock()`, `deleteStock()`

5 Créer la couche controller - Classes `@RestController` pour exposer l'API - Mapper les endpoints avec `@RequestMapping`, `@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`

6 Gestion des DTO et mappage (optionnel) - Créer des DTO pour exposer uniquement les champs nécessaires - Mapper manuellement ou avec MapStruct

7 Gestion des exceptions - Exceptions personnalisées : `ResourceNotFoundException` - `@ControllerAdvice` + `@ExceptionHandler` pour erreurs JSON

8 Sécurité (optionnel) - Spring Security, JWT pour authentification - Contrôle d'accès par rôle (`@PreAuthorize`, `@Secured`)

9 Tests - Unit tests : test des services (`@SpringBootTest`, Mockito) - Integration tests : test des endpoints REST (`MockMvc`)

10 Documentation API - Swagger/OpenAPI (`springdoc-openapi-ui`) - Accès via `/swagger-ui.html` ou `/swagger-ui/index.html`

11 Déploiement - Emballer l'application : `mvn clean package` → jar - Déployer sur serveur ou cloud (Heroku, AWS, Docker) - Configurer la base de données production

✓ **Conseils pratiques** - Commencer par les entités et repositories, puis services et controllers - Tester chaque endpoint avec Postman ou Insomnia - Utiliser Lombok pour réduire le code répétitif - Séparer les DTO des entités pour éviter de renvoyer des infos sensibles